

WEEK 5 & 6

Web Application Security Testing

Ethical Hacking • OWASP Top 10 • Vulnerability Assessment



Objective

Identify & Test Vulnerabilities

Method

Ethical Hacking Techniques

Tools

Burp Suite, OWASP ZAP, Nikto

What We'll Cover Today



01

Environment Setup

DVWA & OWASP Juice Shop installation



02

OWASP Top 10

Key vulnerabilities every tester must know



03

Manual Testing

Hands-on SQL Injection & XSS testing



04

Security Tools

Burp Suite, OWASP ZAP & Nikto usage



05

Findings Report

Documentation & remediation steps

Setting Up the Lab Environment

DVWA Damn Vulnerable Web Application

- 1 Download from GitHub: github.com/digininja/DVWA
- 2 Install XAMPP (Apache + MySQL + PHP)
- 3 Copy DVWA folder to htdocs directory
- 4 Configure config/config.inc.php with DB credentials
- 5 Visit <http://localhost/dvwa> to initialize database
- 6 Default login: admin / password

OWASP Juice Shop

- 1 Visit: owasp.org/www-project-juice-shop
- 2 Install Node.js (v18+) from nodejs.org
- 3 Run: `npm install -g juice-shop`
- 4 Or use Docker: `docker pull bkimminich/juice-shop`
- 5 Start: `node app.js` (port 3000 by default)
- 6 Open browser: <http://localhost:3000>

OWASP Top 10 Vulnerabilities

A01	Broken Access Control	CRITICAL	A02	Cryptographic Failures	HIGH
A03	Injection (SQL, XSS)	CRITICAL	A04	Insecure Design	HIGH
A05	Security Misconfiguration	HIGH	A06	Vulnerable Components	MEDIUM
A07	Auth & Identity Failures	CRITICAL	A08	Insecure Deserialization	HIGH
A09	Security Logging Failures	MEDIUM	A10	Server-Side Request Forgery	HIGH

SQL Injection

**What it is:**

An attacker injects malicious SQL code into input fields to manipulate the database — stealing, modifying, or deleting data without authorization.

⚡ Test Payloads I Used:

```
' OR '1'='1  (always true — bypasses login)

' OR 1=1--  (comment out rest of query)

admin'--  (login as admin, no password)

' UNION SELECT username,password FROM users--
```

⚠ Impact

- Full database dump
- Bypass authentication
- Delete/modify data
- Remote code execution

✅ How I Fixed It / Recommended Remediation:

- Use Prepared Statements / Parameterized Queries: `$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');`
- Input Validation — whitelist allowed characters, reject special SQL chars (', --, ;)
- Principle of Least Privilege — DB user should only have required permissions
- Use an ORM (Object Relational Mapper) that handles escaping automatically



Cross-Site Scripting (XSS)

What it is:

An attacker injects malicious JavaScript into web pages. When victims visit the page, the script runs in their browser — stealing cookies, session tokens, or redirecting users.

Reflected XSS

Malicious script is in the URL. Server reflects it back in response. User clicks a crafted link.

Stored XSS

Script is saved in database (e.g. blog comments). Executes every time page is loaded.

DOM-Based XSS

Vulnerability in client-side JavaScript. No server interaction — all in the browser DOM.

⚡ Test Payloads I Used:

```
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
';alert('XSS');//
<svg onload=alert(document.cookie)>
```

✓ Remediation

- Output encoding (htmlspecialchars)
- Content Security Policy (CSP) header
- Input validation & sanitization
- HttpOnly cookie flag
- Use DOMPurify library for HTML

🔎 Finding: In DVWA (Low Security), the 'Name' field on the XSS Reflected page reflected my input unsanitized. Entering

```
<script>alert("XSS Test")</script> triggered a popup — confirming the vulnerability.
```

Broken Auth & Insecure Deserialization



Broken Authentication

What it is:

Weak or improperly implemented authentication allows attackers to steal credentials, bypass login, or hijack sessions.

How I Tested:

- Tried default credentials: admin/admin, root/root
- Brute-forced login with Burp Suite Intruder
- Checked if session token changes after login
- Tested for account lockout policy
- Looked for password in URL or page source



Insecure Deserialization

What it is:

When apps deserialize data from untrusted sources without validation, attackers can manipulate serialized objects to execute code, escalate privileges, or inject malicious data.

How I Tested:

- Intercepted cookies with Burp Suite
- Decoded base64 session tokens
- Checked for PHP serialize() / unserialize()
- Used ysoserial tool to generate payloads
- Tested Java/PHP deserialization endpoints



Fixes: Use MFA • Strong password policy • Secure session tokens • Never deserialize untrusted input • Use JSON over binary serialization



Security Misconfiguration

The most commonly found vulnerability — default settings, open cloud storage, verbose error messages, and unnecessary features left enabled.



Default Credentials

Admin panels with admin/admin or root/root still enabled



Directory Listing

Web server exposes file structure — /backup, /uploads visible



Error Messages

Stack traces reveal DB structure, file paths, code logic



Unnecessary Services

FTP, Telnet, old PHP/Apache versions running unnecessarily



Missing Security Headers

No X-Frame-Options, CSP, HSTS, X-Content-Type headers



Cloud Misconfiguration

S3 buckets publicly readable, open security group rules



Fix: Harden configurations, disable unused features, change default creds, enable security headers, use Nikto to scan for misconfigurations



Manual Testing Methodology

1	Reconnaissance	Map the app — identify all input fields, forms, URLs, cookies, headers. Note tech stack.
2	Input Mapping	List every user-controllable input: search bars, login forms, URL params, hidden fields.
3	Inject Test Payloads	Try SQL, XSS, command injection payloads in each field. Observe responses carefully.
4	Analyze Response	Look for error messages, changes in output, JS popups, redirects — signs of success.
5	Escalate & Verify	Confirm the vulnerability is real and not a false positive. Measure actual impact.
6	Document Evidence	Screenshot everything — URL, payload, response. Note severity and remediation.



Security Testing Tools



Burp Suite

Traffic Interceptor & Analyzer

Key Features:

- Intercept & modify HTTP/HTTPS requests
- Repeater — replay & tweak requests manually
- Intruder — automated payload injection
- Decoder — encode/decode data (base64, URL)
- Scanner — find vulnerabilities automatically

My Finding:

Used to intercept login requests and modify cookies. Found session token predictability.



OWASP ZAP

Automated Security Scanner

Key Features:

- Automated active & passive scanning
- Spider/crawler to map entire application
- Fuzzer for testing all input points
- Ajax Spider for SPA apps
- Free & open source — great for beginners

My Finding:

Ran automated scan on DVWA — found 12 vulnerabilities including XSS, SQLi, and header issues.



Nikto

Web Server Scanner

Key Features:

- Scans for 6700+ known vulnerabilities
- Checks for outdated server software
- Identifies dangerous files & configs
- Command line: nikto -h http://target
- Fast initial reconnaissance tool

My Finding:

Discovered Apache version, disabled security headers, and test files left on server.



Vulnerability Report — Findings

Vulnerability	Location Found	Severity	Tool Used	Recommended Fix
SQL Injection	/dvwa/login.php (username field)	HIGH	Manual + Burp	Use prepared statements & parameterized queries
Stored XSS	/dvwa/xss_s.php (name input)	HIGH	Manual + ZAP	Output encoding with htmlspecialchars()
Reflected XSS	/dvwa/xss_r.php?name= param	MEDIUM	Manual	Validate & sanitize all reflected user input
Broken Auth	/login page (no logout)	HIGH	Burp Intruder	Implement account logout, MFA, secure sessions
Security Misconfig	/phpinfo.php exposed	MEDIUM	Nikto	Remove sensitive files from production server
Insecure Deser.	/cookie (base64 PHP object)	HIGH	Burp Suite	Avoid PHP unserialize() on user-controlled data
Missing CSP	All pages (HTTP Headers)	LOW	OWASP ZAP	Add Content-Security-Policy response header

Severity:

HIGH

MEDIUM

LOW

SUMMARY

Key Takeaways



Tested 7+ Vulnerabilities

SQL Injection, XSS (Reflected & Stored), Broken Authentication, Insecure Deserialization, and Security Misconfiguration were all successfully identified and documented.



3 Professional Tools Mastered

Gained hands-on experience with Burp Suite for traffic interception, OWASP ZAP for automated scanning, and Nikto for server reconnaissance.



Ethical Testing Principles Applied

All testing was conducted exclusively on intentionally vulnerable lab environments — DVWA & Juice Shop. No real systems were harmed or accessed.

"The goal of ethical hacking is not to break things — it's to find the cracks before the adversaries do."